Assignment 6

Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Information Security

Implementation of columnar transposition cipher for encryption and decryption of messages.

Answer

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void sort_grid(char** grid, char* key) {
    int cols = strlen(key);
    for (int i=0; i<cols-1; i++) {
        for (int j=i+1; j<cols; j++) {
            if (key[j] < key[i]) {
                char** temp = grid[i];
                grid[i] = grid[j];
                grid[j] = temp;

                key[i] = key[i]^key[j];
                key[j] = key[i]^key[j];
                key[i] = key[i]^key[j];
            }
        }
    }
}

char* columnar_cipher(const char* data, const char* key) {
    printf("Initially message : [%s]\n",data);
    int cols = strlen(key);
    int rows = (strlen(data) + cols - 1) / cols;
    char** grid = (char**)calloc(cols,sizeof(char*));

    for (int i=0; i<cols; i++) {
        grid[i] = (char*)calloc(rows+1,sizeof(char));
        for (int j=0; j<rows+1; j++) {
            grid[i][j] = ' ';
        }
        grid[i][rows+1] = '\0';
    }

    for (int i=0; i<strlen(data); i++) {
```

```
        if (data[i] == ' ') continue;
        int y = i / cols;
        int x = i % cols;
        grid[x][y] = data[i];
    }

    printf("\nSetting characters in the grid\n");
    for (int i=0; i<cols; i++) {
        printf("%c : %s\n",key[i],grid[i]);
    }

    char* dup_key = (char*)calloc(cols+1,sizeof(char));
    strcpy(dup_key,key);

    printf("\nSorting grid based on keys\n");
    sort_grid(grid,dup_key);

    printf("\nGrid after sorting\n");
    for (int i=0; i<cols; i++) {
        printf("%c : %s\n",dup_key[i],grid[i]);
    }

    char* codeword = (char*)calloc(cols*(rows+1)+1,sizeof(char));
    codeword[cols*(rows+1)] = '\0';

    for (int i=0; i<(cols*(rows+1)); i++) {
        codeword[i] = grid[i/(rows+1)][i%(rows+1)];
    }

    free(dup_key);

    for (int i = 0; i < cols; i++)
        free(grid[i]);

    free(grid);

    return codeword;
}

int get_sorted_position(const char* key, int col) {
    int pos = 0;

    for (int i = 0; i < strlen(key); i++) {
        if (key[i] < key[col]) pos++;
        if (key[i] == key[col] && i < col) pos++;
    }

    return pos;
}

char* columnar_decipher(const char* codeword, const char* key) {
    printf("Initially codeword : [%s]\n",codeword);
    int cols = strlen(key);
    int rows = (strlen(codeword) + cols - 1) / cols;
```

```
    char** grid = (char**)calloc(cols,sizeof(char*));

    for (int i=0; i<cols; i++) {
        grid[i] = (char*)calloc(rows+1,sizeof(char));
        for (int j=0; j<rows+1; j++) {
            grid[i][j] = ' ';
        }
        grid[i][rows+1] = '\0';
    }

    char* dup_key = (char*)calloc(cols+1,sizeof(char));
    strcpy(dup_key,key);

    char** sorted_grid = calloc(cols,sizeof(char*));
    for (int i = 0; i < cols; i++) {
        sorted_grid[i] = grid[i];
    }

    sort_grid(sorted_grid,dup_key);

    for (int i=0; i<strlen(codeword); i++) {
        if (codeword[i] == ' ') continue;
        int y = i % rows;
        int x = i / rows;
        sorted_grid[x][y] = codeword[i];
    }

    printf("\nSetting characters in the grid\n");
    for (int i=0; i<cols; i++) {
        printf("%c : %s\n",dup_key[i],sorted_grid[i]);
    }

    char* decrypted_message = (char*)calloc(cols*rows+1,sizeof(char));

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            int pos = get_sorted_position(key, j);
            decrypted_message[i * cols + j] = sorted_grid[pos][i];
        }
    }

    decrypted_message[cols*rows] = '\0';

    printf("\nSorting grid based on keys\n");

    printf("\nGrid after sorting\n");
    for (int i=0; i<cols; i++) {
        printf("%c : %s\n",key[i],grid[i]);
    }

    free(dup_key);
    free(sorted_grid);

    for (int i = 0; i < cols; i++)
```

```
        free(grid[i]);

    free(grid);

    return decrypted_message;

}

int main() {
    char* message = "ravish ranjan ";
    char* key = "qwe";

    char* codeword = columnar_cipher(message,key);
    printf("\ncodeword : [%s]\n",codeword);

    char* decrypted_message = columnar_decipher(codeword,key);
    printf("\ndecrypted message : [%s]\n",decrypted_message);

    free(decrypted_message);
    free(codeword);

    return 0;
}
```

Output

```
avish@ravishPC:../is/prog ⎇ (main)> compile -r ./a6.c
Compiled successfully → ./a6.out
Running...
Initially message : [ravish ranjan ]

Setting characters in the grid
q : ri nn
w : asrj
e : vhaa

Sorting grid based on keys

Grid after sorting
e : vhaa
q : ri nn
w : asrj

codeword : [vhaa  ri nn asrj  ]
Initially codeword : [vhaa  ri nn asrj  ]

Setting characters in the grid
e : vhaa
q : ri nn
w : asrj
```

```
Sorting grid based on keys

Grid after sorting
q : ri nn
w : asrj
e : vhaa

decrypted message : [ravish ranjan     ]
ravish@ravishPC:../is/prog ⎇ (main)>
```